

index.htmlについて

index.htmlの役割

index.htmlの役割は、Reactアプリ全体を表示するための場所を1つ用意しておくことである。

Reactは「シングルページアプリケーション」と呼ばれる仕組みで動いている。ページを遷移しても、実際に読み込まれているHTMLファイルはこのindex.html1つだけである。この中にある

がすべての表示の起点になる。

index.jsについて

index.jsの役割

Reactとhtmlを接続するための機動用ファイルである。具体的には、index.htmlの空の

を見つけ出し、そこにApp.jsで作った画面をはめ込むこと(これをマウントと言う)である。

アプリを起動するための設定や、全体の共通設定(ルーティングや全体の状態管理など)も、今後このファイルに追記していく。

index.htmlに描画する部分

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.jsについて

App.jsの役割

App.jsの役割は、画面の見た目(UI)と、入力された時の動き(ロジック)、データの状態(state)を定義することである。

今回の基数変換サイトのロジックや入力欄はここに記述される。これらをReactでは「コンポーネント」と呼ぶ。

{useState}のみ{}な理由

1. {}がない場合(デフォルトインポート)

- 特徴：そのファイル(fromの先)が提供しているメインで提供している代表的なものを受け取る時の書き方
 - ファイルを提供する側は、`export default React;`のように記述されている。
2. {}がある場合(名前付きインポート)
- そのファイル(fromの先)の中に入っている「特定の部品だけを、名前を指定して取得する」ときの書き方。

プログラムの説明

状態(state)の定義：画面を更新するトリガー

```
import { useState } from 'react';

function App() {
  const [binary, setBinary] = useState('');
  const [decimal, setDecimal] = useState('');
  const [error, setError] = useState('');
  :
```

Reactにおける最大のルールは「画面に表示するデータが変化するなら、それはState(状態)として管理しなければならない」という点である。

- `useState`の役割：ただの変数(`let x=0;`など)の値を変えても、Reactは画面を書き換えてくれない。`useState`を使うことで値が更新された瞬間に「画面を再描画(再レンダリング)しろ」という指示がReactに飛ぶ。
- 分割代入：`[binary, setBinary]`という書き方は、JavaScriptの分割代入である。`useState('')`は、「現在の値(初期値は空文字)」と「値を更新するための専用関数」の2つをセットで返す。
 - `binary`：現在の入力値(読み取り専用)
 - `setBinary`：入力値を書き換えるためのスイッチ(この関数を通さないと画面hが更新されない)

ユーザ入力を受け取る：イベントハンドラ

```
const handleChange = (e) => {
  const value = e.target.value;
  setBinary(value);
  :
```

ユーザが入力欄()でキーボードを叩くたびに、この`handleChange`関数が呼び出される。

- 引数`e(Event Object)`：イベント(キー入力など)に関する全ての情報が詰まったオブジェクト。
- `e.target.value`：「イベントが発生した要素(=入力欄)の、現在のテキスト値を取得する。javascriptのメソッド。`e`はeventの略称。
 - ここで使われる`value`はhtmlの要素の`name='value'`というように指定されている。
- `setBinary(value)`：取得した最新のテキストを、ReactのState(`binary`)に保存する。これで画面の入力欄に文字が反映される。

例外処理

```
// 空の時はリセット
if (value === '') {
  setDecimal('');
  setError('');
  return;
}

// 例外処理
if(value !== 0 | value !== 1) {
  setError('エラー：0と1以外が入力されています。');
  setDecimal('');
  return;
}
:
```

ここで使われているvalueは前項でe.target.valueである。

既定変換

```
// 例外処理
if(value !== 0 | value !== 1) {
  setError('エラー：0と1以外が入力されています。');
  setDecimal('');
  return;
}
```

エラーがなければ最後に計算を行う。

- parseInt(文字列, 基数) : javascriptの組み込み関数である。
- toString() : parseIntの結果は「数値(Number型)」である。Reactの入力欄で扱うデータは基本的に「文字列(String型)」であるため、文字列に戻してからState(setDecimal)にセットしている。

透明な箱：React Fragment(<>~</>)

```
return (
  <>
    :
  </>
);
```

Reactの絶対ルールでは、returnでかえす要素(タグ)は、必ず1つの親で囲まれていなければならない。(<></>のことである。)

- 正式名書：React Fragment

- なぜ使うのか：もしここを
で囲むと、ブラウザのHTML い不要な
が出力されてしまい、あとからcssを当てるときに邪魔になることがある。<>/>を利用することで、
「Reactのルールはまもりつつ、実際のhtmlには一歳タグを追加しない(透明な箱として扱う)」という
理想的な状態を作ることができる。

制御されたコンポーネント：入力用タグ

```
<input  
  type="text"  
  value={binary}  
  onChange={handleChange}  
/>
```

ここはReact特有の「制御されたコンポーネント」と呼ばれる非常に重要な仕組みである。

- value={binary}：入力欄の見た目を、ReactのState(binary)に完全に固定している。k ろえにより、「ブラウザが勝手に入力欄の文字を管理する」のではなく、「Reactが入力欄の文字を管理する」状態になる。
- onChange={handleChange}：ユーザがキーボードを叩いた瞬間、handleChange関数が走る。イベントハンドラーを調べることで、さまざまなイベントに対する処理を作ることができる。
- 動きのループ：ユーザが打つ→handleChangeが走る→State(binary)が更新される→value{binary}が更新されて画面の文字が変わる、というループが行われている。

読み取り専用の出力欄:結果表示タグ

```
<input  
  type="text"  
  value={decimal}  
  readOnly  
/>
```

計算された10進数の結果を表示するための入力欄である。

- value={decimal}：Stateで計算された結果(decimal)をここに表示する。
- readOnly：これはhtml標準の属性である。ここには、計算結果が出るだけであり、ユーザに直接文字を打ち込まれては困るため、「読み取り専用」にロックしている。

モジュールのエクスポート

```
export default App;
```

作成したApp関数（コンポーネント）を、他のファイル(今回の場合はindex.js)から読み込めるように公開する。defaultをつけることで、「このファイル(App.js)におけるメインの機能はこれ(App関数)ですよ」と宣言

している。